

Aula 5 — Parte 1

Tipos de Atributos, Similaridade e Dissimilaridade

Objetivos desta sessão

1. Classificar atributos em nominais, ordinais, intervalares e racionais, e reconhecer o que cada tipo permite (ou não) calcular.
2. Explicar por que a distância Euclidiana depende da escala dos atributos numéricos envolvidos.
3. Calcular e comparar distâncias Euclidiana, Manhattan e Jaccard sobre um exemplo pequeno, na mão e em Python.
4. Escolher uma medida de distância coerente com o tipo de atributo em jogo, em vez de aplicar a mesma fórmula a tudo.
5. Reconhecer, numa primeira leitura, por que um único holdout pode dar uma impressão otimista (ou pessimista) de um modelo — o assunto de k-fold e validação cruzada, que retomamos na Semana 9.

1. Retomada: "quão parecidos" ao pé da letra

Desde a Aula 1, você já usa uma noção de distância sem ter formalizado o que ela significa: o K-NN (Aula 1) classifica um exemplo novo olhando para os vizinhos mais **próximos**; o K-Means (Aula 3) agrupa clientes, lojas e estudantes que são **parecidos** entre si. Nas duas aulas, "próximo" e "parecido" eram calculados por uma distância Euclidiana, sem perguntar se ela era a medida certa para aqueles dados.

Hoje formalizamos essa pergunta: que tipo de atributo estamos comparando, e qual noção de distância (ou de similaridade) faz sentido para cada tipo? A resposta importa na prática — usar a métrica errada não dá um erro no código, dá um resultado silenciosamente sem sentido.

2. Tipos de atributos

Um atributo (uma coluna do dataset) pode ser classificado em quatro tipos, que definem quais operações fazem sentido sobre ele:

- **Nominal** — categorias sem nenhuma ordem entre si. Só é possível comparar igualdade: "é a mesma categoria?".
- **Ordinal** — categorias com uma ordem, mas sem uma distância numérica definida entre elas. "maior/menor" faz sentido; "quanto maior" não.
- **Intervalar** — valores numéricos onde diferenças são interpretáveis, mas o zero é arbitrário (não representa "ausência"). Razões entre valores não fazem sentido.
- **Racional** — valores numéricos com um zero absoluto (ausência real da grandeza). Diferenças e razões fazem sentido.

Vamos classificar as colunas do dataset de solicitações de empréstimo, o mesmo da Aula 4 (idade, renda, tempo_emprego, score_credito, cidade):

```
idade, renda, tempo_emprego
```

```
-> RACIONAL: existe zero absoluto (0 ano de idade, R$ 0 de renda,  
    0 ano no emprego atual sao valores reais possiveis) e razoes  
    fazem sentido (60 anos e o dobro de 30; R$ 4000 e o dobro de  
    R$ 2000).
```

score_credito
 -> INTERVALAR: a escala vai de 300 a 850, mas 300 não significa "credito zero" - e só o piso definido pelo modelo de score. Diferenças são interpretáveis (700 - 600 = 100 pontos a mais), mas 700 NÃO é "o dobro" de 350 em nenhum sentido útil.

cidade
 -> NOMINAL: categorias sem ordem natural (SP, RJ, MG, Outra).

Um atributo ordinal, criado sob medida

O dataset não tinha, de origem, nenhum atributo ordinal. Podemos criar um discretizando score_credito em faixas com ordem — "alto" > "médio" > "baixo" — usando `pd.cut(score_credito, bins=[0, 580, 700, 1000], labels=["baixo", "medio", "alto"])`.

O resultado (baixo: 102, médio: 241, alto: 107, em 450 linhas) tem ordem — "alto" é melhor que "médio" — mas não tem uma distância numérica definida entre as faixas: não existe um "quanto" entre baixo e médio, só uma posição relativa.

3. Distância entre atributos numéricos: Euclidiana e Manhattan

Para ver o efeito da escala na prática, tomamos quatro solicitantes completos (sem valores ausentes) do dataset da Aula 4, olhando só para idade e renda:

	idade	renda
S1	47.0	5230.91
S2	52.0	5104.61
S3	44.0	2919.42
S4	28.0	4103.63

A distância Euclidiana entre dois exemplos x e y , com p atributos numéricos, soma o quadrado da diferença em cada atributo e tira a raiz: $d(x,y) = \text{raiz}(\sum (x_i - y_i)^2)$. Sem tratar a escala, o resultado é dominado pela coluna de maior amplitude:

Euclidiana SEM padronizar (escalas originais):

	S1	S2	S3	S4
S1	0.00	126.40	2311.49	1127.44
S2	126.40	0.00	2185.20	1001.27
S3	2311.49	2185.20	0.00	1184.32
S4	1127.44	1001.27	1184.32	0.00

Euclidiana COM padronizacao (StandardScaler):

	S1	S2	S3	S4
S1	0.00	0.57	2.51	2.44
S2	0.57	0.00	2.52	2.88
S3	2.51	2.52	0.00	2.19
S4	2.44	2.88	2.19	0.00

Um cuidado importante

O par mais parecido continua sendo S1-S2 nos dois casos — mas a ordem dos DEMAIS pares muda. Sem padronizar, o segundo par mais próximo é S2-S4 (1001,27); depois de padronizar, passa a ser S3-S4 (2,19), que sem padronizar era só o quarto par mais próximo (1184,32).

É o mesmo problema de escala tratado com StandardScaler na Aula 4 — só que agora ele afeta diretamente quem "parece" mais próximo de quem, não apenas a acurácia de um modelo.

A distância de Manhattan soma as diferenças absolutas em cada eixo, em vez da raiz da soma dos quadrados: $d(x,y) = \sum |x_i - y_i|$. Sobre os mesmos quatro solicitantes, sem padronizar:

Manhattan SEM padronizar:

	S1	S2	S3	S4
S1	0.00	131.30	2314.49	1146.28
S2	131.30	0.00	2193.19	1024.98
S3	2314.49	2193.19	0.00	1200.21
S4	1146.28	1024.98	1200.21	0.00

Os valores são sempre maiores ou iguais aos da Euclidiana para o mesmo par, mas a ordem dos pares é quase idêntica — Manhattan sofre do mesmo problema de escala que a Euclidiana, porque as duas somam diretamente as diferenças de cada coluna.

4. Um atributo nominal: distância de Jaccard

Euclidiana e Manhattan não fazem sentido sobre cidade — "SP" menos "RJ" não é um número. Para um atributo nominal, uma saída é codificar em one-hot (Aula 4) e comparar presença/ausência com a distância de Jaccard, que conta quantas categorias os dois exemplos têm em comum em relação ao total de categorias ativas nos dois:

```
cidade de cada solicitante: ['Outra', 'RJ', 'Outra', 'SP']
categorias (one-hot): ['Outra', 'RJ', 'SP']
```

Distancia de Jaccard (cidade, codificada em one-hot):

	S1	S2	S3	S4
S1	0.0	1.0	0.0	1.0
S2	1.0	0.0	1.0	1.0
S3	0.0	1.0	0.0	1.0
S4	1.0	1.0	1.0	0.0

Com uma única coluna categórica virada one-hot, cada solicitante tem exatamente um "1" — dois solicitantes da mesma cidade têm distância 0 (compartilham a única categoria ativa); de cidades diferentes, distância 1 (nenhuma categoria em comum). S1 e S3 são os dois de "Outra": $Jaccard(S1, S3) = 0,00$.

5. Qual medida usar?

Não existe uma distância "universal" — a escolha depende do tipo de atributo:

Tipo de atributo	Exemplo no dataset	Medida típica
Racional	idade, renda, tempo_emprego	Euclidiana ou Manhattan, sempre padronizados
Intervalar	score_credito	Euclidiana ou Manhattan, após padronizar
Ordinal	faixa_score (baixo/médio/alto)	Codificar respeitando a ordem, depois Manhattan
Nominal	cidade	One-hot + Jaccard (ou Hamming)

Na prática, a maioria dos datasets mistura tipos — como o nosso. O ColumnTransformer da Aula 4 já apontava nessa direção: cada grupo de colunas recebe o tratamento (e, implicitamente, a métrica) adequada ao seu tipo, dentro do mesmo pipeline.

6. Prévia: por que um único holdout pode enganar (k-fold)

Na Aula 4.2 (25/08) comparamos o "caminho correto" de preparo dos dados e chegamos a uma acurácia de 0,8370 no K-NN, usando um único split de 70/30 entre treino e teste. Fica a pergunta: esse número seria parecido se tivéssemos separado o treino/teste de outro jeito?

k-fold responde repetindo o experimento com várias divisões diferentes, em vez de confiar em uma só. A ideia: dividir o dataset em k partes ("folds") de tamanho parecido; treinar em k-1 partes e testar na parte restante; repetir k vezes, revezando qual parte vira o teste; e resumir os k resultados numa média e um desvio-padrão.

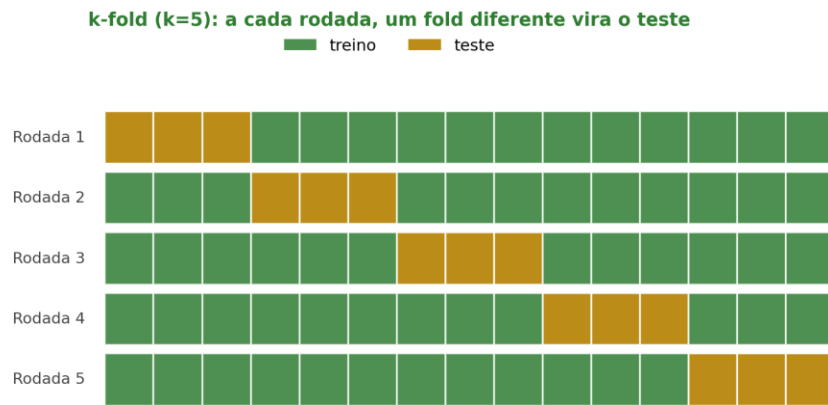


Figura 1 — k-fold com k=5: a cada rodada, um quinto dos dados diferente vira o teste, e os outros quatro quintos viram treino.

Aplicando isso ao mesmo dataset e ao mesmo K-NN da Aula 4.2, com `cross_val_score` e 5 folds:

Acuracia com UM holdout (70/30, o mesmo da Aula 4.2): 0.8370

Acuracia em 5-fold cross-validation (5 splits diferentes):
 scores por fold: [0.7333 0.7222 0.7889 0.8444 0.8333]
 media: 0.7844 desvio-padrao: 0.0500

O que isso muda

O holdout único deu 0,8370 — mas os cinco folds variam de 0,7222 a 0,8444, com média 0,7844. O split de 70/30 usado na Aula 4 calhou de ser um dos mais favoráveis entre os cinco possíveis: sozinho, ele deu uma impressão um pouco otimista do modelo.

k-fold não troca o algoritmo nem os dados — troca a pergunta: em vez de "qual a acurácia neste split?", passa a perguntar "qual a acurácia média, e o quanto ela varia, em vários splits diferentes?". Vamos usar k-fold "pra valer", comparando todos os modelos supervisionados vistos até aqui, na Semana 9 (28/09) — por hoje, fica só o conceito.

O que guardar desta sessão

- Todo atributo é nominal, ordinal, intervalar ou racional — o tipo decide quais operações (igualdade, ordem, diferença, razão) fazem sentido sobre ele.
- Euclidiana e Manhattan, sem padronizar, são dominadas pela coluna de maior escala — o mesmo problema visto com o K-NN na Aula 4, agora afetando diretamente quem "parece" mais próximo.
- Jaccard compara presença/ausência de categorias — a ferramenta certa para atributos nominais codificados em one-hot, onde Euclidiana e Manhattan não fazem sentido.
- Não existe uma distância universal: a escolha da métrica depende do tipo de atributo, e datasets mistos pedem tratamentos diferentes por coluna.
- Um único holdout pode dar uma estimativa otimista ou pessimista da acurácia de um modelo — k-fold repete o experimento em várias divisões para dar uma média mais honesta, com uma noção de quanto ela varia.